MATH 526 Section 001 Lab 10 Spring 2006

Part I

In this part we experiment with the Gauss-Seidel method and sparse matrices.

The Gauss-Seidel iterative method for solving $\mathbf{Ax} = \mathbf{b}$ is defined by the following iterative formula:

$$\mathbf{S}\mathbf{x}^{\text{new}} = \mathbf{b} + \mathbf{T}\mathbf{x}^{\text{old}}$$

where $\mathbf{S}$ is the lower triangular part of $\mathbf{A}$ and $\mathbf{T} = \mathbf{S} - \mathbf{A}$.

Create

$$\mathbf{A} = \begin{bmatrix} 5 & 1 & 1 \\ 1 & 5 & 1 \\ 1 & 1 & 5 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 7 \\ 7 \\ 7 \end{bmatrix}.$$

The exact solution for $\mathbf{Ax} = \mathbf{b}$ is clearly

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

The following commands will compute 5 Gauss-Seidel iterations with initial guess $\mathbf{0}$.

>>x= zeros(3,1)

>>S = tril(A)

The command **tril(A)** picks out the lower triangular part of **A**. What is the command that will pick out the upper triangular part of **A**?

>>T = S − A

>>for k=1:5

>>x = S\(b+T∗x)

>>end

You can see that the sequence of vectors generated by the Gauss-Seidel method is converging to the exact solution very quickly.

>>clear

A sparse matrix is a matrix with very few nonzero components. Typically, the number of nonzero components in a sparse $n \times n$ matrix is of order n. Such matrices are especially suitable for iterative methods. MATLAB can store sparse matrices economically and perform matrix operations involving sparse matrices efficiently.

Create

$$\mathbf{A} = \begin{bmatrix} 4 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

>>B = sparse(A)

This creates the sparse version of **A**. Note that the matrix **B** is presented in terms of the nonzero components and their indices. Use **help** to learn more about **sparse**.

>>full(B)

The command **full** convert a matrix stored in the sparse format to a matrix stored in the standard format.

>>flops(0)

>>A*b

>>flops

>>flops(0)

>>B*b

>>flops

Note that it requires many fewer flops for **Bb** than **Ab**. Thus it is a good idea to store a sparse matrix in the sparse format when you are executing an iterative method.

The sparse format of a matrix is also preserved by matrix operations such as transpose, **diag** or **tril**.

>>B'

>>diag(B)

>>tril(B)

Clear all variables before you work on Part II.